

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



THIẾT KẾ VI MẠCH (CO3098)

Bài Thực Hành 3

Logic Equivalence Check

Lớp: L04 - Nhóm: 08 - Học Kỳ HK252

Ngày nộp: 27/04/2026

GVHD: Thầy Phạm Kiều Nhật Anh

Sinh Viên	MSSV
Phạm Công Võ	2313946
Phạm Minh Đức	2311079
Nguyễn Trần Như Phước	2212719
Nguyễn Thanh Tùng	2112711

TP.Hồ Chí Minh, Tháng 04/2026



Mục lục

1	Giới thiệu	2
1.1	Mục tiêu Lab	2
1.2	Công cụ sử dụng	2
2	Môi trường thực hiện	3
2.1	Thông tin sinh viên và đường dẫn làm việc	3
2.2	Cấu trúc thư mục	3
2.3	Tạo Symbolic Link	3
3	Phần 1 - LEC Cơ bản	4
3.1	Quy trình thực hiện	4
3.2	Kết quả LEC - Phần 1	5
3.3	Cảnh báo trong quá trình chạy LEC	6
4	Phần 2 - Debug Non-equivalent Point	8
4.1	Tạo Bug trong Netlist	8
4.2	Kết quả LEC - Phần 2	8
4.3	Debug Non-equivalent Point	9
4.4	Giải pháp khắc phục	10
5	Tổng kết	10

1 Giới thiệu

1.1 Mục tiêu Lab

Bài thực hành số 3 tập trung vào quy trình **kiểm tra tương đương logic** (*Logic Equivalence Checking - LEC*) giai đoạn quan trọng trong chu trình thiết kế vi mạch số, xác minh rằng Netlist sau tổng hợp (*Gate-Level Netlist*) tương đương về mặt logic với mô tả RTL gốc, sử dụng công cụ **Cadence Conformal LEC**.

Quy trình LEC trong bài thực hành được chia thành hai phần:

1. **Phần 1 - LEC cơ bản:** Thực hiện kiểm tra tương đương giữa RTL (Golden) và Netlist (Revised), xác nhận kết quả *Equivalent*.
2. **Phần 2 - Debug Non-equivalent Point:** Cố tình tạo bug trong Netlist, chạy lại LEC, quan sát và debug các điểm Non-equivalent thông qua GUI (Mapping Manager, Diagnosis Manager, Schematics Viewer).

Sau khi hoàn thành bài thực hành, sinh viên có khả năng:

- ▶ Xây dựng môi trường LEC, tạo symbolic link từ kết quả synthesis sang thư mục `lec_env` và viết script `lec.tcl`.
- ▶ Thực thi Conformal LEC từ command-line thông qua script `go lec` và đọc hiểu kết quả trong log file và GUI.
- ▶ Phân tích kết quả *Mapped points*, *Compared points* và xác định trạng thái *Equivalent* / *Non-equivalent*.
- ▶ Sử dụng **Mapping Manager**, **Diagnosis Manager** và **Schematics Viewer** để định vị và xác định nguyên nhân gây ra Non-equivalent point.
- ▶ Đề xuất giải pháp khắc phục lỗi logic: re-synthesize hoặc Engineering Change Order (ECO).

1.2 Công cụ sử dụng

Công cụ		Phiên bản	Vai trò
Cadence Conformal		24.10	Kiểm tra tương đương logic giữa RTL và Netlist, thực thi qua script TCL (<code>lec -64 -dofile lec.tcl</code>).
Conformal GUI		24.10	Giao diện đồ họa xem Mapping Manager, Diagnosis Manager và Schematics Viewer để debug Non-equivalent points.
Cadence Genus		22.1	Tổng hợp logic tại Lab 2, sinh ra file Netlist <code>bound_flasher_m.v</code> làm đầu vào cho LEC.
Standard Cell Library	Cell	gsclib045	Thư viện cell chuẩn 45 nm (<code>slow.lib</code>), được load vào Conformal để giải nghĩa các cell trong Netlist.

Bảng 1: Danh sách công cụ sử dụng trong Bài thực hành 3

Mối quan hệ giữa các công cụ trong quy trình được tóm tắt như sau: **Xcelium** (Lab 1) → RTL .v → **Genus** (Lab 2) → Netlist .v → **Conformal** (Lab 3) → LEC Pass/Fail.

2 Môi trường thực hiện

2.1 Thông tin sinh viên và đường dẫn làm việc

Thông tin	Giá trị
Group	104group8
Student ID	2313946
Thư mục gốc	~/vlsi/2313946/work/
RTL (Golden)	synthesis_env/Genus_BoundFlasher/RTL/bound_flasher.v
Netlist (Revised)	synthesis_env/Genus_BoundFlasher/LAB1/ outputs_Apr10-16:47:59/bound_flasher_m.v
Thư mục LAB3 LEC	lec_env/

Bảng 2: Thông tin sinh viên và đường dẫn làm việc

2.2 Cấu trúc thư mục

```
1 ~/vlsi/2313946/work/
2 |-- synthesis_env/
3 |   '-- Genus_BoundFlasher/
4 |     |-- RTL/
5 |       |-- bound_flasher.v      <- RTL (Golden)
6 |     |-- LIB/
7 |       |-- slow.lib             <- Thu vien cell
8 |     '-- LAB1/
9 |       '-- outputs_Apr10-16:47:59/
10 |         '-- bound_flasher_m.v  <- Netlist (Revised)
11 '-- lec_env/                    <- Thu muc lam viec LAB3
12   |-- bound_flasher.v    -> symlink -> RTL
13   |-- bound_flasher_m.v  -> symlink -> Netlist (hoac copy that)
14   |-- slow.lib           -> symlink -> Library
15   |-- lec.tcl             <- Script cau hinh Conformal
16   |-- go_lec              <- Script thuc thi
17   '-- lec.log             <- Log sinh ra sau khi chay
```

2.3 Tạo Symbolic Link

Thay vì copy file, tạo symbolic link từ `synthesis_env` vào `lec_env` để tiết kiệm dung lượng và đảm bảo luôn dùng file mới nhất:

```
1 cd ~/vlsi/2313946/work/lec_env
2 ln -sf ../synthesis_env/Genus_BoundFlasher/RTL/bound_flasher.v
3 ln -sf ../synthesis_env/Genus_BoundFlasher/LAB1/outputs_Apr10-16:47:59/bound_flasher_m.v
4 ln -sf ../synthesis_env/Genus_BoundFlasher/LIB/slow.lib
```

Kiểm tra link thành công bằng lệnh `ll` – ba file phải hiển thị có mũi tên `->` chỉ đến đường dẫn gốc.

3 Phần 1 - LEC Cơ bản

3.1 Quy trình thực hiện

1. **Tạo file lec.tcl:** File script cấu hình trình tự thực thi của Conformal:

```
1 set_log_file lec.log -replace
2 read_library slow.lib -lib -revised
3 read_design bound_flasher.v -verilog -golden
4 read_design bound_flasher_m.v -verilog -revised
5 set_mapping_method -name only
6 set_system_mode lec
7 map_key_points
8 add_compared_points -all
9 compare
```

2. **Tạo file go_lec:** Script thực thi, source license và khởi động Conformal:

```
1 #!/bin/bash -f
2 cd /home/share_file/cadence
3 source add_path
4 source add_license
5 cd -
6 lec -64 -dofile ./lec.tcl &
```

Sau đó cấp quyền thực thi và chạy:

```
1 chmod +x ./go_lec
2 ./go_lec
```

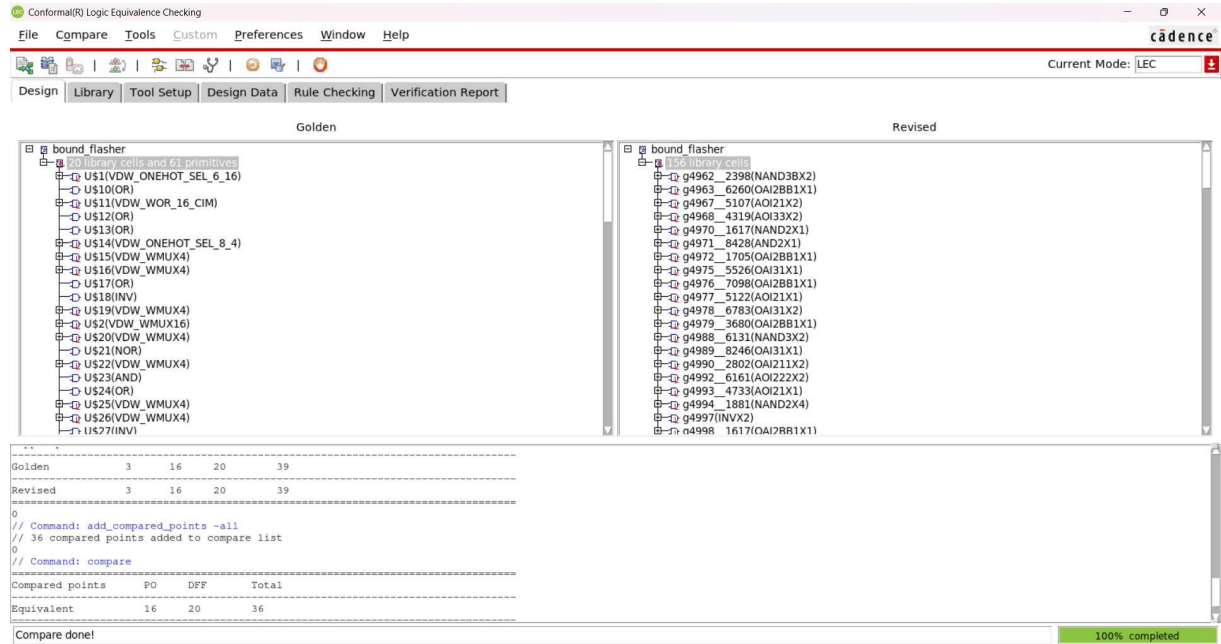
Bảng 3 giải thích ý nghĩa từng lệnh trong lec.tcl:

Lệnh	Ý nghĩa
set_log_file lec.log -replace	Ghi log ra file lec.log, ghi đè nếu đã tồn tại
read_library slow.lib -lib -revised	Load thư viện cell để giải nghĩa các gate trong Netlist
read_design ...-golden	Đọc RTL làm thiết kế chuẩn (Golden)
read_design ...-revised	Đọc Netlist cần kiểm tra (Revised)
set_mapping_method -name only	Mapping key points theo tên signal
set_system_mode lec	Chuyển sang chế độ kiểm tra tương đương
map_key_points	Tạo bản đồ mapping giữa Golden và Revised
add_compared_points -all	Thêm tất cả điểm vào danh sách so sánh
compare	Thực hiện so sánh và xuất kết quả

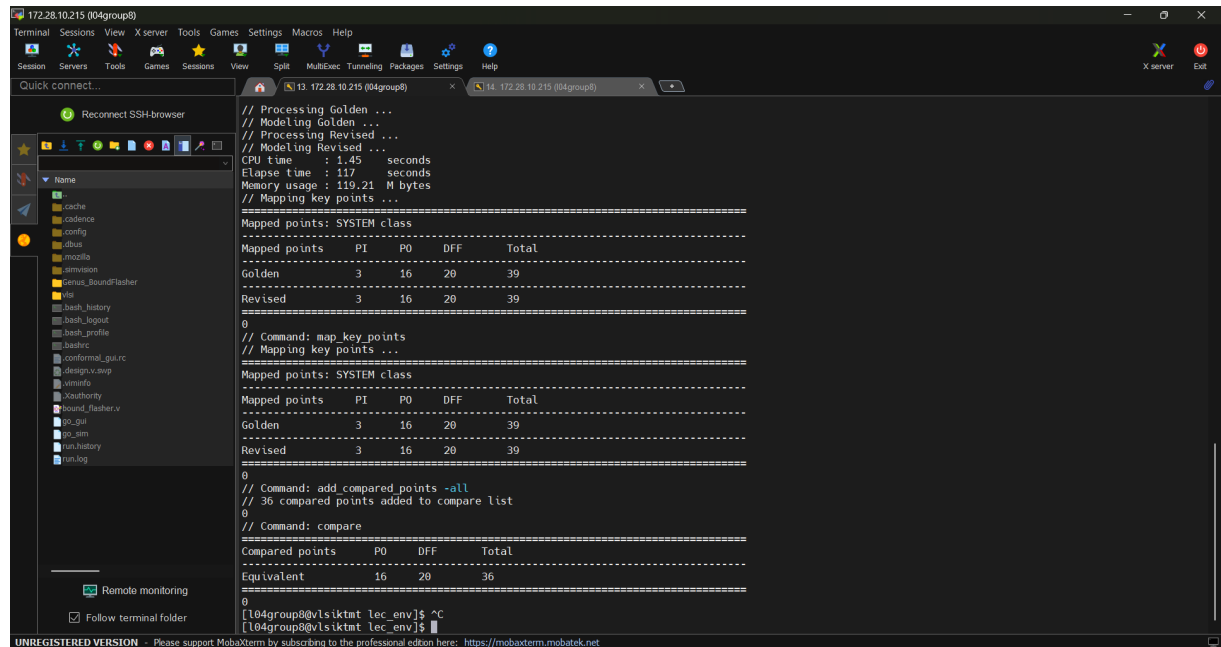
Bảng 3: Giải thích các lệnh trong file lec.tcl

3.2 Kết quả LEC - Phần 1

Sau khi chạy `./go_lec`, GUI Conformal tự động mở và thực thi script. Kết quả được hiển thị trong cửa sổ terminal phía dưới GUI và trong file `lec.log`:



Hình 1: Giao diện GUI Conformal - Phần 1: LEC Pass (Equivalent 36/36, Compare done!)



Hình 2: Log terminal - Quá trình chạy `lec.tcl` thành công, không có Error

Kết quả Phần 1:

Thông số	PO	DFF	Total
Mapped points (Golden)	16	20	39
Mapped points (Revised)	16	20	39
Compared points	16	20	36
Equivalent	16	20	36
Non-equivalent	0	0	0

Bảng 4: Kết quả LEC Phần 1 - 36/36 điểm Equivalent

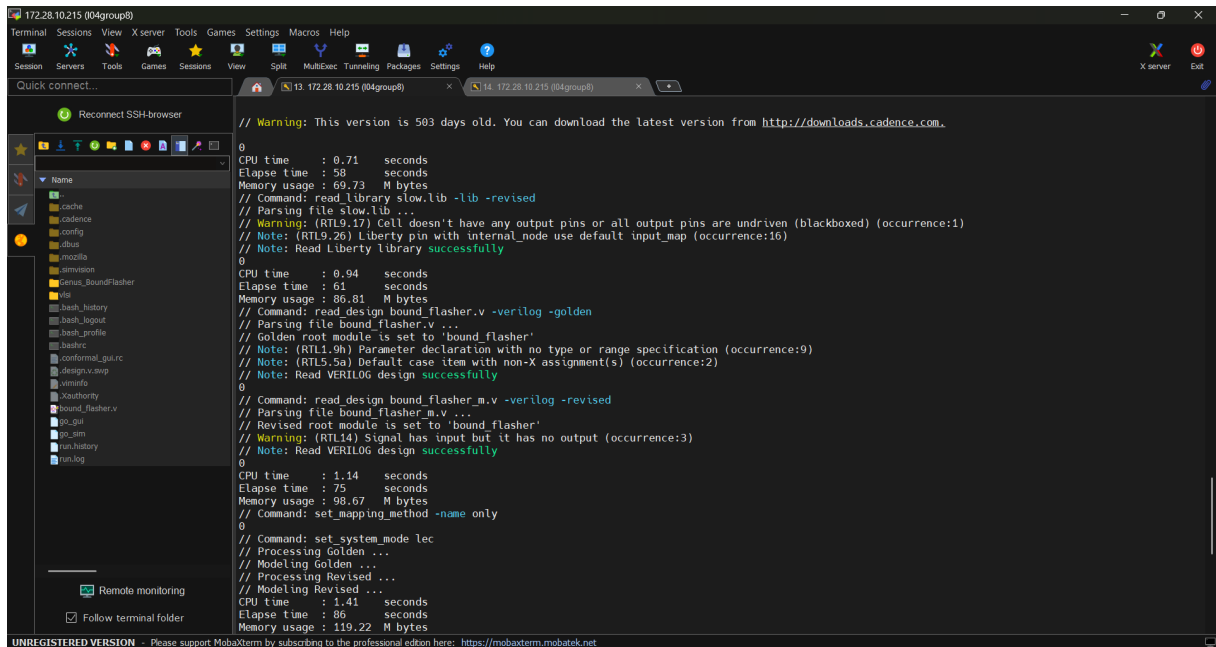
Nhận xét:

Toàn bộ 36 điểm so sánh đều **Equivalent** - Netlist sau synthesis tương đương hoàn toàn với RTL gốc về mặt logic. Điều này xác nhận quá trình synthesis tại Lab 2 không làm thay đổi chức năng thiết kế.

3.3 Cảnh báo trong quá trình chạy LEC

Trong quá trình thực thi `lec.tcl`, Conformal phát sinh một số thông báo cần lưu ý. Trong đó, cảnh báo quan trọng nhất là **Warning RTL14**:

```
1 // Warning: (RTL14) Signal has input but it has no output (occurrence:3)
```



Hình 3: Warning RTL14 xuất hiện trong log terminal khi chạy `lec.tcl`

Warning RTL14 là gì?

Cảnh báo này cho biết có **3 tín hiệu nội bộ** trong Netlist được kết nối vào pin input của một cell nhưng **không có cell nào drive** (tạo ra) tín hiệu đó - tức là wire có đầu nhận nhưng không có đầu phát.

Vì sao xuất hiện?

- ▶ **Synthesis tối ưu hóa logic:** Genus đôi khi loại bỏ (optimize away) một số gate trong quá trình tối ưu, khiến tín hiệu đầu ra của gate đó biến mất nhưng wire kết nối vẫn còn trong Netlist.
- ▶ **Don't-care conditions trong RTL:** File `bound_flasher.v` có một số điều kiện `default case` với phép gán non-X (như *Note RTL5.5a* trong log). Sau synthesis, những nhánh này có thể tạo ra dangling wire không có driver.
- ▶ **Tín hiệu tie-off ngầm:** Một số tín hiệu được gán giá trị cố định bên trong thư viện cell mà không có wire rõ ràng trong Netlist - Conformal đọc vào thấy thiếu driver.

Mức độ ảnh hưởng:

Cảnh báo này **không ảnh hưởng** đến kết quả LEC. Conformal chỉ phát sinh warning ở giai đoạn `read_design`, không dừng quá trình so sánh. Toàn bộ 36 điểm vẫn **EQUIVALENT** như trình bày tại Bảng 4.

Cách khắc phục:

Phương pháp	Mô tả
Bổ sung default trong RTL	Thêm default đầy đủ trong tất cả case statement của <code>bound_flasher.v</code> , đảm bảo mọi tín hiệu đều có driver trong mọi trạng thái. Đây là giải pháp triệt để nhất.
Thêm constraint synthesis	Thêm <code>set_db / .resolve_undriven_nets tie_low</code> vào <code>run.tcl</code> trước khi chạy Genus, để tool tự tie-off các net không có driver trong Netlist.
Suppress trong Conformal	Thêm lệnh <code>set_undefined_cell black_box -both</code> vào <code>lec.tcl</code> để Conformal bỏ qua warning này khi đọc design. Phù hợp khi không muốn sửa RTL hoặc re-synthesize.
Chấp nhận warning	Không can thiệp - vì LEC vẫn PASS và đây là hiện tượng bình thường sau synthesis trong môi trường thực hành.

Bảng 5: Các phương pháp khắc phục Warning RTL14

4 Phần 2 - Debug Non-equivalent Point

4.1 Tạo Bug trong Netlist

Để thực hành quy trình debug, cố tình tạo lỗi trong Netlist bằng cách thay thế một cell **INV** (Inverter) bằng **BUF** (Buffer):

```
1 # Buoc 1: Copy that Netlist (không dùng symlink để tránh ảnh hưởng file gốc)
2 rm -rf bound_flasher_m.v
3 cp -rf ../synthesis_env/Genus_BoundFlasher/LAB1/outputs_Apr10-16:47:59/bound_flasher_m.v
4 ./
5 # Buoc 2: Tìm dòng có INV cell
6 grep -n "INV" bound_flasher_m.v | head -10
7
8 # Buoc 3: Đổi INVX2 -> BUF2 tại dòng 104 (chỉ đổi tên cell, giữ nguyên port .Y)
9 sed -i '104s/INVX2/BUF2/' bound_flasher_m.v
10
11 # Xác nhận đã sửa đúng
12 sed -n '104p' bound_flasher_m.v
```

Kết quả sau khi sửa tại dòng 104:

Trạng thái	Nội dung dòng 104
Trước khi sửa	INVX2 g4997(.A(n_130), .Y(n_146));
Sau khi sửa	BUF2 g4997(.A(n_130), .Y(n_146));

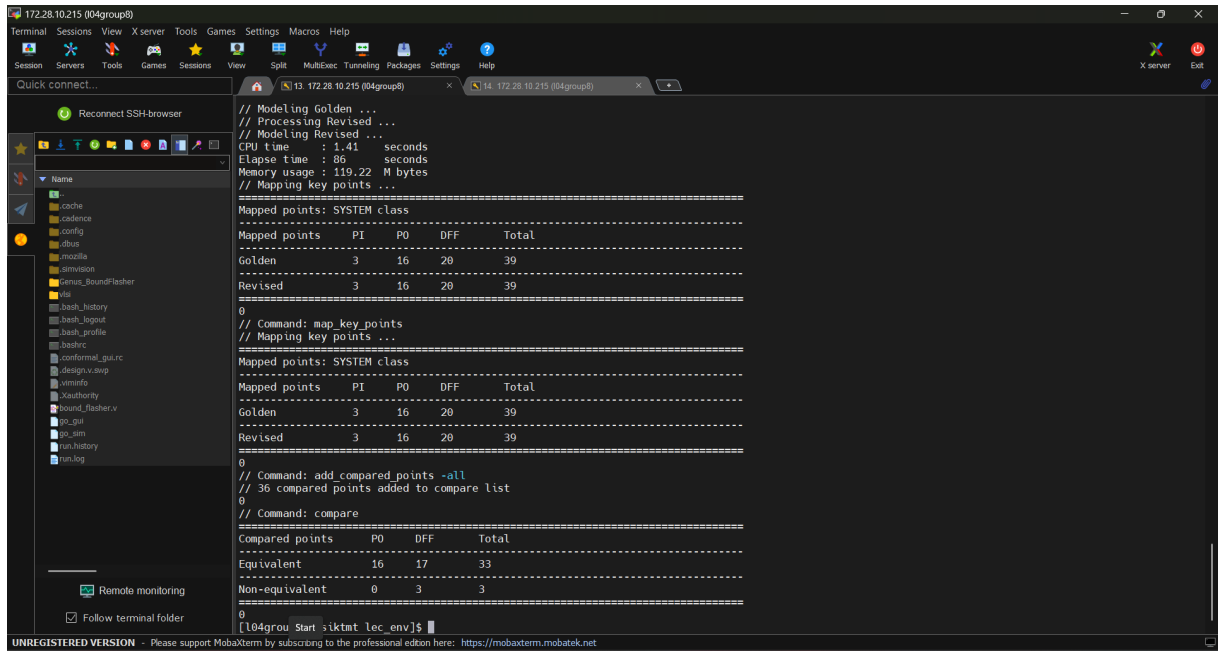
Bảng 6: So sánh netlist trước và sau khi tạo bug

Lý do tạo ra Non-equivalent:

- ▶ **INVX2** (Inverter): tín hiệu vào bị **đảo ngược** trước khi ra - input 0 → output 1.
- ▶ **BUF2** (Buffer): tín hiệu vào được **giữ nguyên** - input 0 → output 0.
- ▶ Kết quả: tín hiệu **n_146** có giá trị **ngược** so với Golden, các DFF phía sau bị ảnh hưởng tạo ra Non-equivalent points.

4.2 Kết quả LEC - Phần 2

Sau khi chạy lại `./go_lec` với Netlist đã bị sửa, Conformal phát hiện các điểm không tương đương:



Hình 4: Log terminal - Phần 2: Xuất hiện 3 Non-equivalent points

Thông số	PO	DFF	Total
Compared points	16	17	33
Equivalent	16	17	33
Non-equivalent	0	3	3

Bảng 7: Kết quả LEC Phần 2 - 3 điểm Non-equivalent

4.3 Debug Non-equivalent Point

Quy trình debug được thực hiện qua GUI của Conformal theo các bước:

- Mapping Manager - Filter Non-Equivalent:** Mở Mapping Manager → Class → Disable All → chọn **Non-Equivalent**. Danh sách lọc còn lại 3 DFF bị lỗi: `state_reg[0]`, `state_reg[1]`, `state_reg[3]`.
- Diagnosis Manager - Xác định Error Candidate:** Right-click vào `state_reg[0]` → **Diagnose**. Cửa sổ Diagnosis Manager hiển thị **Error Candidate** với độ tin cậy 1.00:

```

1 Error Candidate:
2   (1.00) BUF 104   g4997/U$1    <- Root cause (do tin cay 100%)
3   (1.00) INV 92    g5134/U$1
4   (1.00) NAND 93   g5111_7410/U$1
  
```

Conformal tự xác định **BUF g4997/U\$1** (cell vừa bị sửa từ `INVX2` → `BUFx2`) là nguyên nhân với độ tin cậy **100%**.

- Schematics Viewer - Quan sát logic cone:** Trong Diagnosis Manager → chọn **Schematics**. Hai cửa sổ schematic mở song song:

- Bên trái - **Revised** (Netlist): endpoint = 1
- Bên phải - **Golden** (RTL): endpoint = 0

Right-click pin endpoint → **Fan-in Cone** → **Open** để trace ngược tìm root cause.

4. **Xác định Root Cause:** Hover chuột lên gate lỗi trong schematic → tooltip hiển thị số dòng trong source code. Double-click gate → file `bound_flasher_m.v` mở ra, trở đúng dòng 104 chứa `BUF2 g4997`.

4.4 Giải pháp khắc phục

Sau khi xác định root cause là cell **BUF2 g4997** tại dòng 104 (đáng lẽ phải là **INV2**), có hai hướng giải quyết:

Giải pháp	Mô tả
Re-synthesize	Chạy lại synthesis từ RTL gốc với Genus để tạo ra Netlist mới không có lỗi. Đây là giải pháp an toàn nhất khi bug xuất phát từ lỗi synthesis.
ECO (Engineering Change Order)	Sửa trực tiếp Netlist: đổi <code>BUF2</code> → <code>INV2</code> tại dòng 104, sau đó chạy lại LEC để xác nhận đã fix. Phù hợp khi chỉ có một số ít cell bị lỗi và không muốn re-synthesize toàn bộ.

Bảng 8: Các giải pháp khắc phục *Non-equivalent point*

5 Tổng kết

Thông số	Phần 1 - LEC Pass	Phần 2 - Debug
Netlist sử dụng	Gốc (symlink)	Đã sửa (copy thật)
Compared points	36	33
Equivalent	36	33
Non-equivalent	0	3
Bug được tạo	Không	INV2 → BUF2 tại dòng 104
Root cause	-	g4997/U\$1 (BUF)
Kết quả	PASS	FAIL (3 Non-equiv)

Bảng 9: So sánh kết quả Phần 1 và Phần 2

Kết luận: Bài thực hành đã minh họa thành công hai kịch bản quan trọng trong quy trình LEC:

- **Phần 1:** Netlist sau synthesis tương đương hoàn toàn với RTL - xác nhận quá trình synthesis tại Lab 2 chính xác.

- ▶ **Phần 2:** Công cụ Conformal LEC có khả năng phát hiện lỗi logic cụ thể (cell sai loại) với độ chính xác cao (confidence 1.00), giúp định vị root cause nhanh chóng thông qua GUI mà không cần phân tích thủ công toàn bộ Netlist.